

Machine Learning – An Introduction

Artificial Intelligence (AI) is the ability of machines to perform tasks that normally require human thinking, such as decision-making, problem-solving, and understanding language.

Machine Learning (ML) is a part of AI that helps machines learn from data and improve automatically without being programmed again and again.

Example:

A chatbot that answers questions is an AI system. If the chatbot improves its answers by learning from previous conversations, then it is using Machine Learning.

Evolution of Machine Learning

Machine Learning evolved from traditional programming to solve problems that are difficult to handle using fixed rules.

In traditional programming, programmers write instructions in a programming language. These instructions work on data and produce answers.

Basic flow of traditional programming:

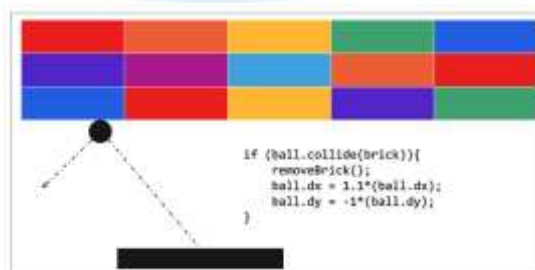
Data + Rules → Answers

Traditional Programming

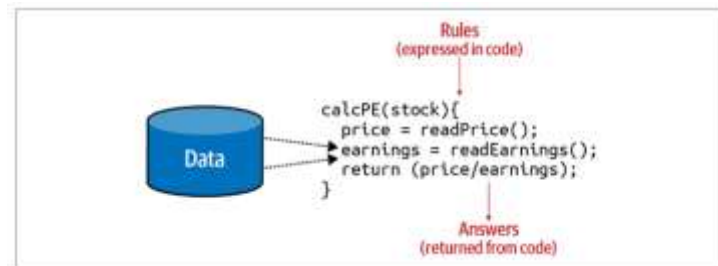
Traditional programming uses predefined rules written by humans. The computer follows these rules step by step.

For example:

- In the game Breakout, code controls the movement of the ball, scoring, and winning or losing conditions. When the ball hits a brick, the brick disappears and the ball changes direction.



- In financial services, code can calculate the P/E ratio by dividing a company's stock price by its earnings.



The program cannot learn by itself. Any changes must be made manually by the programmer.

Limitations of Traditional Programming

Traditional programming is useful for simple and rule-based tasks, but it struggles when:

- Data becomes very large
- Problems become complex
- Rules are difficult to define clearly

For example:

- Face recognition
- Spam email detection
- Predicting customer behavior

Writing exact rules for these tasks is difficult



Introduction of Machine Learning

Machine Learning solves these limitations by allowing computers to learn from data instead of following only fixed instructions.

In Machine Learning:

- Data is given to the system
- Algorithms learn patterns from the data

- The system improves automatically over time

Basic flow of Machine Learning:

Data + Learning Algorithm → Predictions/Answers

Advantages of Machine Learning

Machine Learning can:

- Learn from experience
- Improve performance automatically
- Handle complex data
- Identify hidden patterns

Limitations of Traditional Programming

Traditional programming has been used for many years and works by applying rules to data to generate answers.

This method works well only when programmers can clearly define rules for every situation. If the problem becomes too complex, writing rules becomes very difficult or impossible.

Activity Detection Example

Fitness trackers are a good example of the limitations of traditional programming.

Suppose we want to detect whether a person is walking, running, or biking using their speed.

Detecting Walking

A simple rule can be:

- If speed is low, the person is probably walking.

Detecting Running

The rules can be extended:

- If speed is less than 4 mph, the person is walking.
- If speed is more than 4 mph, the person is running.

Detecting Biking

The algorithm can again be expanded:

- Higher speed values may indicate biking.

Even though this method is simple and not always accurate, it still works for basic activities.



```
if(speed<4){
    status=WALKING;
}
```



```
if(speed<4){
    status=WALKING;
} else {
    status=RUNNING;
}
```



```
if(speed<4){
    status=WALKING;
} else if(speed<12){
    status=RUNNING;
} else {
    status=BIKING;
}
```

Problem with Complex Activities

The problem appears when trying to detect activities like golfing.

In golfing:

- A person may walk for some time
- Stop frequently
- Swing the club
- Walk again

It becomes very difficult to write exact rules to identify this activity using only speed or simple conditions.

At this point, traditional programming reaches its limitation.



From Programming to Machine Learning

Machine Learning solves this problem by changing the approach.

Traditional Programming:

Data + Rules → Answers

Machine Learning:

Data + Answers → Rules

Instead of programmers creating all the rules manually, the computer learns the rules from data.

How Machine Learning Works

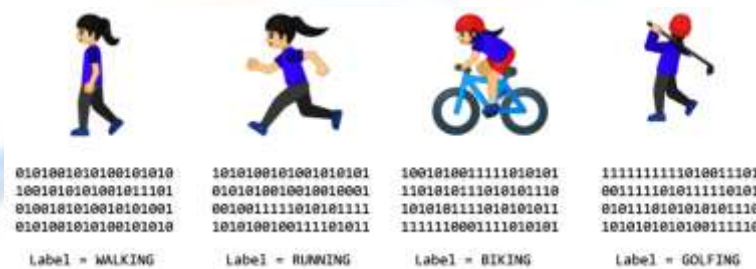
In Machine Learning:

- Large amounts of data are collected
- Data is labeled with correct answers

- Algorithms study patterns in the data
- The machine learns rules automatically

For activity detection:

- Sensor data such as speed, heart rate, and location is collected
- Activities like walking, running, and biking are labeled
- The system learns patterns for each activity



This allows the machine to recognize activities more accurately, even complex ones like golfing.



Role of Programmers

In traditional programming:

- Programmers write rules manually.

In Machine Learning:

- Programmers collect and label data
- Algorithms automatically discover the rules

Machine Learning and Artificial Intelligence

Machine Learning is an important part of Artificial Intelligence (AI).

AI focuses on making computers behave intelligently like humans.

Machine Learning helps computers learn from examples, similar to how humans learn through experience.

Using Machine Learning, computers can:

- Recognize images
- Understand speech
- Read text

- Detect activities
- Make predictions

Getting Started with Machine Learning

Machine Learning works by finding patterns in data. Instead of writing rules manually, we provide examples to the computer, and it learns the relationship automatically.

Example of Learning from Data

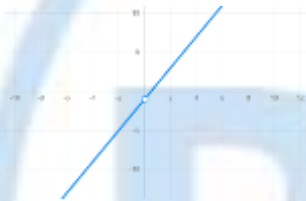
Consider the following values:

$X = -1, 0, 1, 2, 3, 4$

$Y = -3, -1, 1, 3, 5, 7$

By observing the pattern, we can see that:

$$Y = 2X - 1$$



As the value of X increases, the value of Y also changes according to this formula.

Machine Learning models try to discover this relationship automatically from the data.

Machine Learning Process

The basic process of Machine Learning is:

- Provide input data
- Provide correct output values
- Train the model
- The model learns the pattern
- Use the trained model to make predictions

Using Scikit-learn

Scikit-learn is a popular Python library used for Machine Learning. It provides simple tools for building and training models.

Importing Libraries

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

- NumPy is used for handling numerical data.
- LinearRegression is used to create a simple learning model.

Preparing the Data

```
X = np.array([-1, 0, 1, 2, 3, 4]).reshape(-1, 1)
```

```
Y = np.array([-3, -1, 1, 3, 5, 7])
```

The input values are stored in X and the output values are stored in Y.

.reshape(-1, 1) converts the data into the format required by Scikit-learn.

Creating the Model

```
model = LinearRegression()
```

This creates a Linear Regression model that will learn the relationship between X and Y.

Training the Model

```
model.fit(X, Y)
```

The fit() function trains the model using the given data.

During training:

- The model studies the input and output values
- It tries to discover the mathematical relationship
- It adjusts itself to reduce prediction errors

Making Predictions

```
print(model.predict([[10]]))
```

After training, the model predicts the value of Y when X is 10.

The expected answer is close to:

$$Y = 2(10) - 1 = 19$$

The prediction may not be exactly 19 because Machine Learning models learn approximately from data.

Understanding Prediction

In Machine Learning, the term prediction means estimating an output based on learned patterns.

The model does not memorize answers. Instead, it learns relationships and uses them to predict new values.

Linear Regression

Linear Regression is one of the simplest Machine Learning algorithms.

It is called linear because the relationship between X and Y forms a straight line.

It tries to find a straight-line relationship between input and output values.

General equation of Linear Regression:

$$Y = WX + B$$

Where:

- W represents weight or slope
- B represents bias or intercept
- X is the input value
- Y is the predicted output

For this example:

- W becomes close to 2
- B becomes close to -1

The model learns the values of W and B during training.

What Does W Mean?

W controls how much Y changes when X changes.

In our example:

$X = -1, 0, 1, 2, 3, 4$

$Y = -3, -1, 1, 3, 5, 7$

We can notice that whenever X increases by 1, Y increases by 2.

So, the model learns that:

$W \approx 2$

This means the slope of the line is close to **2**.

What Does B Mean?

B is the starting value of Y when X is 0.

From the data:

- When $X = 0$, $Y = -1$

So the model learns:

$$B \approx -1$$

This means the line crosses the Y-axis at -1 .

After training, the model learns an equation close to:

$$Y = 2X - 1$$

The values may not be exactly 2 and -1 because Machine Learning models learn from limited data and predictions are approximate.

If there are multiple input features, the equation becomes:

$$Y = W_1X_1 + W_2X_2 + W_3X_3 + B$$

Example of Multiple Linear Regression

Suppose we want to predict a student's final marks based on:

- Study hours
- Attendance
- Assignment score

The equation can be:

$$Y = W_1X_1 + W_2X_2 + W_3X_3 + B$$

Where:

- (X_1) = Study hours
- (X_2) = Attendance percentage
- (X_3) = Assignment marks
- (Y) = Final marks

Assume:

- ($W_1 = 2$)
- ($W_2 = 0.5$)
- ($W_3 = 1$)
- ($B = 5$)

If a student has:

- Study hours = 4
- Attendance = 80
- Assignment marks = 10

The value **B=5** is called the bias or intercept. Even if Study hours, attendance and assignment marks are 0 predicted marks would still start from 5.

Then:

$$Y = (2 \times 4) + (0.5 \times 80) + (1 \times 10) + 5 = 63$$

So, the predicted final marks are 63.

Code Example:

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Input data
# [Study Hours, Attendance, Assignment Marks]
X = np.array([[2, 60, 5], [3, 70, 6], [4, 80, 8], [5, 85, 9], [6, 90, 10]]) # X is input features
Y = np.array([45, 52, 63, 72, 80]) # Y contains final marks.

# Create model
model = LinearRegression()

model.fit(X, Y) # trains the model using the data.

# Predict final marks
prediction = model.predict([[4, 80, 10]])

print("Predicted Final Marks:", prediction[0])
```

```
# Display learned weights and bias
```

```
print("Weights:", model.coef_) # model.coef_ gives the weights
```

```
print("Bias:", model.intercept_) # model.intercept_ gives the bias
```

Model Types in Scikit-learn

Learning Type	Model Type	Explanation
Supervised	Classification	Used to predict categories or classes such as yes/no, spam/not spam, pass/fail
Supervised	Regression	Used to predict numerical values such as marks, salary, or house prices
Unsupervised	Clustering	Used to group similar data together without labels
Unsupervised	Dimensionality Reduction	Used to reduce the number of features while keeping important information
Supervised	Ensemble	Combines multiple models to improve accuracy and performance, for example Random Forest, Gradient Boosting, and AdaBoost

What is a Classifier?

A classifier is a Machine Learning model that predicts categories or classes.

For example, it can predict whether a message is “Spam” or “Not Spam” based on features like words, links, or message length.

This type of learning is called Supervised Learning because the model learns using labeled data.

Features and Labels

- Features are the input characteristics of the data.

Examples: Message length, No of links, keywords etc

- Labels are the categories the model predicts.

Examples: spam , not spam

Learning Algorithm

A learning algorithm finds patterns in the training data and creates a model that can predict labels for new data.

Note:

Classifiers are commonly used in Supervised Learning(Spam Detection), Clustering models are commonly used in Unsupervised Learning(Customer Grouping).

Scikit-learn Training Models

Learning Type	Model Type	Model Name	Purpose	Example Use	Basic Syntax	Import Syntax
S	Classification	Logistic Regression	Predicts simple categories like Yes/No or True/False	Spam detection	LogisticRegression()	from sklearn.linear_model import LogisticRegression
S	Classification	Decision Tree	Makes predictions using rule-based decisions	Loan approval	DecisionTreeClassifier()	from sklearn.tree import DecisionTreeClassifier
S	Classification	Random Forest	Uses many decision trees together for better prediction accuracy	Fraud detection	RandomForestClassifier()	from sklearn.ensemble import RandomForestClassifier
S	Classification	Support Vector Machine (SVM)	Separates data into different categories	Face recognition	SVC()	from sklearn.svm import SVC
S	Classification	K-Nearest Neighbors (KNN)	Predicts output using nearby similar data	Recommendation systems	KNeighborsClassifier()	from sklearn.neighbors import KNeighborsClassifier
S	Regression	Linear Regression	Predicts numerical values using a straight-line relationship	House price prediction	LinearRegression()	from sklearn.linear_model import LinearRegression
S	Regression	Polynomial Regression	Predicts numerical values with curved relationships	Sales growth prediction	PolynomialFeatures()	from sklearn.preprocessing import PolynomialFeatures

S	Regression	Ridge Regression	Improves Linear Regression and reduces overfitting	Market analysis	Ridge()	from sklearn.linear_model import Ridge
S	Regression	Lasso Regression	Improves regression and removes less important features	Data optimization	Lasso()	from sklearn.linear_model import Lasso
S	Ensemble	Gradient Boosting	Builds models step by step to improve prediction accuracy	Risk prediction	GradientBoostingClassifier()	from sklearn.ensemble import GradientBoostingClassifier
S	Ensemble	AdaBoost	Improves predictions by focusing on previous mistakes	Classification improvement	AdaBoostClassifier()	from sklearn.ensemble import AdaBoostClassifier
U	Clustering	K-Means Clustering	Groups similar data into clusters automatically	Customer segmentation	KMeans()	from sklearn.cluster import KMeans
U	Clustering	DBSCAN	Creates clusters based on dense groups of data	Anomaly detection	DBSCAN()	from sklearn.cluster import DBSCAN
U	Dimensionality Reduction	Principal Component Analysis (PCA)	Reduces the number of features while keeping important information	Image compression	PCA()	from sklearn.decomposition import PCA

Steps to Create a Basic Machine Learning Model in Scikit-learn

Supervised learning

Step 1:

Import Scikit-learn, NumPy, and other required libraries.

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

Step 2:

Store input data in X and output data in Y.

```
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
```

```
Y = np.array([2, 4, 6, 8, 10])
```

Step 3:

Create the Machine Learning model using syntax like LinearRegression().

```
model = LinearRegression()
```

Step 4:

Train the model using the fit() function.

```
model.fit(X, Y)
```

Step 5:

Use the predict() function to predict outputs.

```
prediction = model.predict([[6]])
```

```
print(prediction)
```

Step 6: Check Results

View predictions, accuracy, weights, or model performance.

```
print("Weight:", model.coef_)
```

```
print("Bias:", model.intercept_)
```

Unsupervised Learning

Step 1:

Import Scikit-learn, NumPy, and other required libraries.

```
import numpy as np
```

```
from sklearn.cluster import KMeans
```

Step 2:

Store the input data in X.

```
X = np.array([[1, 2], [1, 4], [5, 8], [8, 8]])
```

Step 3:

Create the Unsupervised Learning model.

```
model = KMeans(n_clusters=2)
```

Step 4:

Train the model using the fit() function.

```
model.fit(X)
```

Step 5:

Use the model to identify clusters.

```
prediction = model.predict(X)
```

```
print(prediction)
```

Step 6: Check Results

View cluster labels or cluster centers.

```
print(model.cluster_centers_)
```

```
print(model.labels_)
```

A Complete End-to-End Basic ML Model (Titanic Dataset)

Step 1: Import Libraries

Import all required libraries.

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

Step 2: Load Dataset

Load the dataset using Pandas.

```
data = pd.read_csv("data.csv")
```

Step 3: Preprocess Data

Handle missing or unnecessary data.

Example:

```
data = data.dropna()
```

This removes rows containing missing values.

Step 4: Feature Selection

Select input features (X) and output labels (Y).

```
X = data[['Age', 'Fare']]
```

```
Y = data['Survived']
```

- X contains input features
- Y contains output labels

Step 5: Choose Values

Select the columns or values needed for prediction.

Example chosen features:

- Age
- Fare

Target label:

- Survived

Step 6: Split Data

Split the dataset into training and testing data.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

Step 7: Scale Values

Scaling helps normalize data values.

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

fit_transform()

- Learns scaling values
- Transforms training data

transform()

Transforms testing data using previously learned scaling values.

Step 8: Create Model

Create the Machine Learning model.

```
model = LogisticRegression()
```

Step 9: Train Model

Train the model using training data.

```
model.fit(X_train, Y_train)
```

Step 10: Make Predictions

Predict outputs using test data.

```
prediction = model.predict(X_test)
```

Step 11: Evaluate Model

Check model accuracy.

```
accuracy = accuracy_score(Y_test, prediction)
```

```
print("Accuracy:", accuracy)
```

Tensors

A tensor is a multidimensional data structure used in TensorFlow.

Tensors are similar to arrays or matrices and are used to store and process data in Machine Learning and Deep Learning.

A tensor can contain:

- Numbers
- Arrays
- Multidimensional data

TensorFlow uses tensors as the main data format for computations.

Types of Tensors

Tensor Type	Dimensions	Example
Scalar Tensor	0D (single value)	5
Vector Tensor	1D (list of values)	[1, 2, 3]
Matrix Tensor	2D (rows and columns)	[[1, 2], [3, 4]]
3D Tensor	3D data structure	[[[1,2],[3,4]], [[5,6],[7,8]]]

What is TensorFlow?

TensorFlow is an open-source Machine Learning framework used to create, train, and deploy ML models.

It provides built-in tools and algorithms that help developers build Machine Learning and Artificial Intelligence applications without writing complex mathematical logic manually.

Main Uses of TensorFlow

TensorFlow is used for:

- Creating ML models
- Training models using data
- Making predictions (inference)
- Deploying models on web, mobile, and cloud platforms

Training and Inference

Training

Training is the process where the model learns patterns from data.

Example:

A model is trained using many images of cats and dogs to learn their differences.

Inference

Inference is the process of using the trained model to make predictions on new data.

Example:

After training, the model predicts whether a new image contains a cat or a dog.

Ways to Create Models in TensorFlow

TensorFlow provides different methods to build models:

- Writing code manually
- Using built-in estimators
- Using Keras API

Keras is the most commonly used and beginner-friendly API in TensorFlow.

TensorFlow Components

TensorFlow Data Services

Used for:

- Loading datasets
- Processing raw data
- Managing training data

TensorFlow Lite

Used to run models on:

- Android
- iOS
- Embedded systems (small computer systems built inside devices)

TensorFlow.js

Used to run ML models in:

- Web browsers
- Node.js applications

TensorFlow Lite Micro (TFLM)

Used for TinyML(perform intelligent tasks without needing internet or powerful computers on very small or low power devices) and microcontrollers(very small computers placed inside electronic devices to control specific operations).

Hardware Support

TensorFlow supports:

- CPU
- GPU
- TPU

Using GPUs and TPUs helps train models faster.

Installing TensorFlow

Install TensorFlow

```
pip install tensorflow
```

Install CPU Version

```
pip install tensorflow-cpu
```

Checking TensorFlow Version

```
import tensorflow as tf
```

```
print(tf.__version__)
```

This displays the installed TensorFlow version.

Linear Regression using TensorFlow and Keras API

```
import tensorflow as tf
```

```
import numpy as np
```

```
from tensorflow.keras import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
model = Sequential([Dense(units=1, input_shape=[1])])  
model.compile(optimizer='sgd', loss='mean_squared_error')  
xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)  
ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float)  
model.fit(xs, ys, epochs=500)  
print(model.predict([10.0]))
```

Understanding Neural Networks

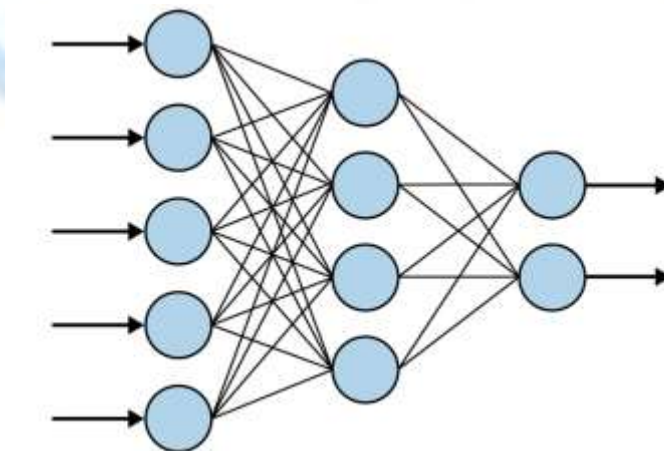
A neural network is inspired by the human brain. It consists of:

- **Neurons**
- **Layers**
- **Connections between neurons**

A typical neural network contains:

1. **Input Layer**
2. **Hidden Layer(s)**
3. **Output Layer**

Each neuron receives input, performs calculations, and passes output to the next neuron.



Importing Libraries

a) TensorFlow

import tensorflow as tf

- TensorFlow is a powerful **machine learning framework** developed by Google.
- It provides tools for:
 - Deep Learning
 - Neural Networks
 - AI applications

b) NumPy

import numpy as np

- NumPy is a Python library for numerical computations.
- It helps store and process data efficiently using arrays.

c) Sequential

from tensorflow.keras import Sequential

- Sequential is used to build a neural network **layer by layer in sequence**.
- It is the simplest model architecture in Keras.
- Beside sequential, TensorFlow also provides Functional API(Used for more complex neural networks) and Model Subclassing(Fully Customised neural network)

d) Dense Layer

from tensorflow.keras.layers import Dense

- A Dense layer means:
 - Every neuron is connected to every neuron in the next layer.
- It is also called a **fully connected layer**.

Other Types of Layers in TensorFlow

Layer Type	Purpose	Commonly Used In
------------	---------	------------------

Dense Layer	Fully connected neurons used for general learning	Basic neural networks, classification, regression
Conv2D Layer	Used for image processing and computer vision	Image recognition, object detection
MaxPooling Layer	Reduces image size and keeps important features	Convolutional Neural Networks (CNNs)
Flatten Layer	Converts multidimensional data into a single list	Connecting CNN layers to Dense layers
Dropout Layer	Prevents overfitting by randomly ignoring some neurons during training	Deep Learning models
Embedding Layer	Used for text and Natural Language Processing	Chatbots, text classification
LSTM Layer	Used for sequence and time-series data	Speech recognition, language translation
SimpleRNN Layer	Processes sequential data step by step	Sequence prediction, time-series analysis

Creating the Neural Network

```
model = Sequential([Dense(units=1, input_shape=[1])])
```

This line creates the neural network.

Sequential Model

Sequential(...)

- Creates a stack of layers.
- Layers are added one after another.

Dense Layer

```
Dense(units=1, input_shape=[1])
```

units=1

- The layer contains **1 neuron**.

input_shape=[1]

- Input data has only **one value**.

- Example:
 - $X = 1$
 - $X = 2$
 - $X = 10$

Each input contains a single number.

```
model = Sequential([  
    Dense(units=4, input_shape=[2]),  
    Dense(units=1)  
])
```

Here:

- **First Dense layer:**
 - Hidden layer
 - Contains 4 neurons
- **Second Dense layer:**
 - Output layer
 - Contains 1 neuron

If there is only one layer in the model, it act as output layer.

Mathematical Meaning of the Neuron

The neuron learns:

$$Y = WX + B$$

Where:

- (W) = Weight
- (B) = Bias

The network will try to learn values close to:

$$Y = 2X - 1$$

So ideally:

- $(W = 2)$
- $(B = -1)$

Compiling the Model

```
model.compile(optimizer='sgd', loss='mean_squared_error')
```

This prepares the model for learning.

Optimizer

```
optimizer='sgd'
```

SGD means:

Stochastic Gradient Descent

It helps the model improve its guesses step by step.

Helps improve the model by reducing errors

What Does the Optimizer Do?

Suppose the model guesses:

$$Y = 10X + 10$$

This is incorrect.

The optimizer:

1. Measures the error
2. Adjusts weights and bias
3. Makes a better guess

This process repeats many times.

Loss Function

```
loss='mean_squared_error'
```

The loss function measures **how wrong the prediction is**.

Mean Squared Error (MSE)

It calculates:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Mean Error Squared

Why Squaring?

Squaring:

- Removes negative signs
- Penalizes large errors more heavily

Pattern in Data

$$Y = 2X - 1$$

The network must discover this pattern.

Training the Model

```
model.fit(xs, ys, epochs=500)
```

This is the learning stage.

Epochs

epochs=500

An epoch means:

- One complete pass through the training data.

So:

- The model studies the data **500 times**.

How Learning Happens

Step 1: Initial Guess

The model starts with random values.

Example:

$$Y = 5X + 8$$

Step 2: Calculate Error

The model compares predictions with correct answers.

Step 3: Compute Loss

Loss function measures error.

Step 4: Optimizer Updates Values

SGD adjusts:

- Weight (W)
- Bias (B)

Step 5: Repeat

This cycle repeats for 500 epochs.

Over time:

- Error decreases
- Predictions improve

Loss Reduction

Initially:

- Loss is large

After many epochs:

- Loss becomes very small

Example:

Epoch	Loss
1	3.2868
10	0.9682
500	0.000026

This means the model has learned the relationship successfully.

Note:

- Scikit-learn uses built-in ML algorithms directly.
- TensorFlow builds and trains neural networks to learn patterns.